

Roadmap — Projet Ruban Rose

Classification histopathologique du cancer du sein · Dataset BreakHis · Master IA & Data 2026

Objectif : Développer un outil de classification du cancer du sein à partir d'images histopathologiques. Le projet compare deux architectures de pointe (**EfficientNetV2** et **ViT**) et propose une analyse fine des performances par grossissement et une étude d'explicabilité (XAI).

Structure du projet

```
ruban-rose/
├── 01_EDA_BreakHis.ipynb           ← Phase 1 : exploration complète
├── 02_Preprocessing_Split.ipynb    ← Phase 2 : split + augmentation
├── 03_Modeling_EfficientNetV2.ipynb ← Phase 3a : CNN état de l'art
├── 04_Modeling_ViT.ipynb          ← Phase 3b : Transformer
├── 05_Evaluation_Interpretability.ipynb ← Phase 4 & 5 : métriques + XAI
├── README.md
├── requirements.txt
└── slides/
```

Livrables attendus

Livable	Contenu
01_EDA_BreakHis.ipynb	Statistiques par grossissement, sous-types, visualisations
02_Preprocessing_Split.ipynb	Split par patient, augmentation, DataLoader
03_Modeling_EfficientNetV2.ipynb	Entraînement + HPO Optuna
04_Modeling_ViT.ipynb	Entraînement + HPO Optuna
05_Evaluation_Interpretability.ipynb	Métriques, Grad-CAM, Attention Maps, t-SNE
README.md	Contexte, données, algorithmes, veille, conclusion
Slides	Présentation orale
Repo GitHub public	Nommé ruban-rose

Phase 1 — Veille & Exploration du dataset (EDA)

Durée estimée : 2-3 jours · **Notebook :** 01_EDA_BreakHis.ipynb

1.1 Veille scientifique

- Contexte médical : comprendre l'IDC (Invasive Ductal Carcinoma), la biopsie H&E, le gold standard diagnostic
- Lire 2-3 papiers de référence :
 - [Deep Residual Learning for Image Recognition \(ResNet\)](#)
 - [EfficientNetV2 : Smaller Models and Faster Training](#)
 - [An Image is Worth 16×16 Words \(ViT\)](#)
- Identifier les benchmarks existants sur BreakHis (performances état de l'art attendues : ~90-94% accuracy)

1.2 Audit approfondi du dataset

- Télécharger BreakHis : [Kaggle - BreakHis](#)
- **Volumétrie globale** :

Métrique	Valeur
Images totales	7 909 PNG (700×460 px)
Patients distincts	82
Classe Bénin	2 480 images (~31%)
Classe Malin	5 429 images (~69%)
Grossissements	40×, 100×, 200×, 400×
Sous-types tumoraux	8 (4 bénins, 4 malins)

- **Analyse granulaire** — à faire dans le notebook :
 - Distribution des images par grossissement (volume équilibré ?)
 - Distribution des 8 sous-types : Adénose, Fibroadénome, Tumeur Phyllodes, Adénome Tubulaire (bénins) / Carcinome Ductal, Lobulaire, Mucineux, Papillaire (malins)
 - Vérification des doublons ou images corrompues
 - Visualiser **minimum 20 images** par classe et par grossissement

💡 **Lien avec la recherche** : cette phase correspond exactement à une "analyse de corpus" en sciences humaines — on caractérise l'échantillon avant tout traitement, pour identifier les biais potentiels.

🌀 Phase 2 — Prétraitement & Split rigoureux

Durée estimée : 1-2 jours · Notebook : [02_Preprocessing_Split.ipynb](#)

2.1 Split train / val / test via Fold officiel (Fold 1) ← point critique

⚠️ **Règle d'or : On utilise le split officiel du dataset.** Au lieu d'un split aléatoire global (`random_state=42`), on utilise le split officiel du dataset contenu dans `Folds.csv`. Ce fichier définit, pour chaque fold (1 à 5), quels fichiers appartiennent au train ou au test via la colonne `grp`. Cela garantit la **reproductibilité** pour comparer avec l'état de l'art, et assure que les **patients sont déjà strictement séparés** entre train et test. Il faut toutefois extraire un set de validation à partir du train set, en isolant les patients pour éviter toute fuite de données (data leakage).

🔗 **Structure de Folds.csv** (Kaggle – ambarish/breakhis) :

filename	fold	grp
SOB_M_DC-14-2523-100-008.png	1	train
SOB_B_A-14-22549AB-40-001.png	1	test

```
import os
import re
import pandas as pd
from sklearn.model_selection import train_test_split

# — 1. Charger le fichier de folds officiel —————
df_folds = pd.read_csv('./data/ambarish/Folds.csv')

# — 2. Filtrer pour ne garder que le Fold 1 —————
df_fold1 = df_folds[df_folds['fold'] == 1].copy()

# — 3. Extraire le patient_id depuis le filename —————
# Exemple : SOB_M_DC-14-2523-100-008.png → patient_id = "14-2523"
# Pattern : SOB_[M/B]_[TYPE]-[patient_id]-[magnification]-[slice].png
def extract_patient_id(filename):
    """Extrait l'identifiant patient du nom de fichier BreakHis."""
    match = re.search(r'SOB_[MB]_[A-Z]+-(\d+-\d+)-\d+', filename)
    return match.group(1) if match else None

df_fold1['patient_id'] = df_fold1['filename'].apply(
    lambda f: extract_patient_id(os.path.basename(f))
)

# — 4. Séparer Train initial et Test selon la colonne officielle 'grp' —————
train_df_initial = df_fold1[df_fold1['grp'] == 'train'].copy()
test_df = df_fold1[df_fold1['grp'] == 'test'].copy()

print(f"Train initial : {len(train_df_initial)} images | "
      f"Patients : {train_df_initial['patient_id'].nunique()}")
print(f"Test : {len(test_df)} images | "
      f"Patients : {test_df['patient_id'].nunique()}")

# — 5. Créer un set de validation (Early Stopping) depuis le train officiel —
# IMPORTANT : split par patient_id → pas d'images du même patient dans train ET val
train_patients = train_df_initial['patient_id'].unique()

# 15% des patients du train → validation
train_patients_final, val_patients = train_test_split(
    train_patients,
    test_size=0.15,
    random_state=42 # seed uniquement pour ce sous-split val, pas pour
    train/test
)
```

```
# — 6. Filtrer les DataFrames finaux —————
train_df =
train_df_initial[train_df_initial['patient_id'].isin(train_patients_final)].copy()
val_df =
train_df_initial[train_df_initial['patient_id'].isin(val_patients)].copy()

print(f"\n— Résultat final —")
print(f"train_df : {len(train_df)} images | {train_df['patient_id'].nunique()} patients")
print(f"val_df   : {len(val_df)} images   | {val_df['patient_id'].nunique()} patients")
print(f"test_df  : {len(test_df)} images  | {test_df['patient_id'].nunique()} patients")

# — 7. Vérification anti data-leakage —————
assert len(set(train_df['patient_id']) & set(val_df['patient_id'])) == 0, \
    "⊖ Data leakage train/val !"
assert len(set(train_df['patient_id']) & set(test_df['patient_id'])) == 0, \
    "⊖ Data leakage train/test !"
assert len(set(val_df['patient_id']) & set(test_df['patient_id'])) == 0, \
    "⊖ Data leakage val/test !"
print("☑️ Aucun data leakage détecté.")
```

💡 **Ce split est défini UNE SEULE FOIS et ne change plus jamais.** Le `random_state=42` ne sert qu'à stabiliser le sous-split de validation (15 % des patients train). Le split train/test est entièrement déterminé par le fichier officiel `Folds.csv`, colonne `grp`.

2.2 Pipeline de transformation

```
from torchvision import transforms

# Train : avec augmentation
transform_train = transforms.Compose([
    transforms.Resize((224, 224)),          # requis pour ViT et EfficientNet
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(                 # simule la variabilité H&E entre
labos
        brightness=0.2, contrast=0.2, saturation=0.1
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# Val / Test : sans augmentation
transform_eval = transforms.Compose([
```

```
transforms.Resize((224, 224)),
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

2.3 Gestion du déséquilibre des classes (31% / 69%)

Approche recommandée : combiner **(a)** + **(b)**

(a) WeightedRandomSampler — batches équilibrés à chaque époque

```
import torch
from torch.utils.data import WeightedRandomSampler

labels_train = torch.tensor(train_dataset.targets)
class_counts = torch.bincount(labels_train)
class_weights = 1.0 / class_counts.float()
sample_weights = class_weights[labels_train]

sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)
```

(b) Weighted Cross-Entropy Loss — pénalise plus les erreurs sur la classe rare

```
import torch.nn as nn

class_weights_loss = torch.tensor([0.69, 0.31]).to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights_loss)
```

(c) Focal Loss (*bonus, si résultats insuffisants*)

Focal Loss = $-\alpha \times (1 - pt)^\gamma \times \log(pt)$ Paramètres recommandés : $\gamma=2$, $\alpha=[0.25, 0.75]$

🌀 Phase 3 — Modélisation & Optimisation avec Optuna

Durée estimée : 1 semaine · Notebooks : 03_... et 04_...

3.1 Architecture 1 — EfficientNetV2 (CNN)

Principe : extrait des caractéristiques locales en hiérarchie (contours → formes → structures cellulaires). EfficientNetV2 est plus rapide à entraîner qu'EfficientNetB3 avec de meilleures performances.

```
import timm # bibliothèque recommandée pour les architectures modernes

model = timm.create_model('tf_efficientnetv2_s', pretrained=True, num_classes=2)

# Phase 1 : Feature Extraction (backbone gelé)
for param in model.parameters():
    param.requires_grad = False
for param in model.classifier.parameters():
    param.requires_grad = True

# Phase 2 : Fine-tuning (dégel progressif des dernières couches)
for param in model.blocks[-2:].parameters(): # 2 derniers blocs
    param.requires_grad = True
```

3.2 Architecture 2 — Vision Transformer (ViT)

Principe : découpe l'image en patches 16×16, puis utilise des mécanismes d'attention globale. Contrairement au CNN, il voit toute l'image d'un coup → très adapté à la structure tissulaire.

```
model_vit = timm.create_model('vit_base_patch16_224', pretrained=True,
num_classes=2)

# Le ViT nécessite plus de données pour le fine-tuning
# → commencer par geler tout sauf la tête (MLP head)
for param in model_vit.parameters():
    param.requires_grad = False
for param in model_vit.head.parameters():
    param.requires_grad = True
```

3.3 Hyperparameter Optimization (HPO) avec Optuna

```
import optuna

def objective(trial):
    lr_head = trial.suggest_float('lr_head', 1e-4, 1e-2, log=True)
    lr_backbone = trial.suggest_float('lr_backbone', 1e-6, 1e-4, log=True)
    batch_size = trial.suggest_categorical('batch_size', [16, 32, 64])
    dropout = trial.suggest_float('dropout', 0.2, 0.6)
    weight_decay = trial.suggest_float('weight_decay', 1e-5, 1e-3, log=True)
    optimizer_name = trial.suggest_categorical('optimizer', ['Adam', 'AdamW',
'SGD'])

    # ... entraîner le modèle
    return val_f1_score # métrique cible pour Optuna

study = optuna.create_study(direction='maximize') # maximiser le F1
study.optimize(objective, n_trials=50)
print("Meilleurs hyperparamètres :", study.best_params)
```

Hyperparamètres à explorer :

Hyperparamètre	Plage suggérée
Learning Rate (tête)	1e-4 → 1e-2
Learning Rate (backbone)	1e-6 → 1e-4
Batch Size	16, 32, 64
Dropout	0.2 → 0.6
Weight Decay	1e-5 → 1e-3
Optimizer	Adam, AdamW, SGD
Epochs max	20 → 50 avec early stopping

3.4 Bonnes pratiques d'entraînement

```
# Seeds reproductibles – à mettre au tout début de chaque notebook
import torch, numpy as np, random

torch.manual_seed(42)
np.random.seed(42)
random.seed(42)
torch.backends.cudnn.deterministic = True

# Early stopping basé sur val_loss
# Patience = 7 époques → sauvegarder best_model.pth

# Scheduler cosinus
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=20, eta_min=1e-6
)
```

🌀 Phase 4 — Évaluation granulaire & métriques

Notebook : [05_Evaluation_Interpretability.ipynb](#)

4.1 Métriques de performance

⚠ **Un modèle trivial qui prédit toujours "Malin" obtient 69% d'accuracy sans rien apprendre.**

Métrique	Formule	Priorité
Accuracy	(VP+VN) / Total	Référence uniquement
Precision	VP / (VP+FP)	Secondaire

Métrique	Formule	Priorité
Recall (Malin)	$VP / (VP + FN)$	★ Cible ≥ 0.85
F1-score macro	$2 \times P \times R / (P + R)$	★ Métrique principale
ROC-AUC	Aire sous courbe ROC	Comparaison modèles
PR-AUC	Aire sous courbe P/R	★★ Recommandé sur données déséquilibrées

Un **faux négatif** = cancer non détecté. La patiente rentre sans diagnostic. Le **Recall** est donc la métrique la plus critique cliniquement.

4.2 Analyse par grossissement ← apport Gemini

Idée clé : les 4 grossissements ne donnent pas les mêmes informations biologiques.

Grossissement	Échelle observée	Difficulté attendue
40×	Architecture tissulaire globale	Plus facile (patterns larges)
100×	GroupeS cellulaires	Moyen
200×	Cellules individuelles	Moyen
400×	Détails nucléaires fins	Plus difficile (patches petits)

À faire dans le notebook :

- Calculer les métriques séparément pour chaque grossissement
- Générer une matrice de confusion par grossissement
- Conclure : est-ce que l'échelle tissulaire (40×) ou cellulaire (400×) est plus informative ?

4.3 Analyse des confusions inter-classes (classification multi-classes — bonus)

- Matrice de confusion 8×8 sur les sous-types tumoraux
- Quels sous-types sont confondus entre eux ? (ex : Adénose vs Fibroadénome)
- Discuter des implications cliniques de ces erreurs spécifiques

🌀 Phase 5 — Explicabilité (XAI) & Interprétabilité

Notebook : [05_Evaluation_Interpretability.ipynb](#) (suite) Objectif : ouvrir la "boîte noire" et rassurer le corps médical.

5.1 Grad-CAM / Grad-CAM++ pour EfficientNetV2 (CNN)

```
from pytorch_grad_cam import GradCAM, GradCAMPlusPlus
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

# Cibler la dernière couche convolutive
cam = GradCAM(model=model_efficientnet,
```



```
target_layers=[model_efficientnet.blocks[-1]])

targets = [ClassifierOutputTarget(1)] # classe Malin
grayscale_cam = cam(input_tensor=img_tensor, targets=targets)
visualization = show_cam_on_image(img_rgb, grayscale_cam[0])
```

À analyser :

- Le modèle s'active-t-il sur les **noyaux cellulaires denses** (zones biologiquement pertinentes) ?
- Ou sur des **artefacts** (bruit, bordures de lame) → signe de biais
- Analyser **minimum 5 faux négatifs** (cancer non détecté) et les commenter

5.2 Attention Maps pour ViT ← apport Gemini

Grad-CAM n'est pas adapté aux Transformers. On utilise **Attention Rollout** à la place.

```
# pip install vit-explain ou utiliser pytorch_grad_cam avec ViT wrapper
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

# Wrapper ViT pour pytorch-grad-cam
from pytorch_grad_cam.utils.reshape_transforms import vit_reshape_transform

cam_vit = GradCAM(
    model=model_vit,
    target_layers=[model_vit.blocks[-1].norm1],
    reshape_transform=vit_reshape_transform
)

grayscale_cam = cam_vit(input_tensor=img_tensor, targets=targets)
```

À analyser :

- Quels **patches spatiaux** de l'image ont le plus d'attention lors de la décision ?
- Comparer visuellement avec les heatmaps Grad-CAM d'EfficientNetV2

5.3 Visualisation de l'espace latent (t-SNE / UMAP) ← apport Gemini

Idée : extraire les embeddings de la dernière couche de chaque modèle et les projeter en 2D. Permet de **voir** si les classes Bénin / Malin sont bien séparées dans l'espace représentationnel.

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Extraire les embeddings (avant la couche de classification)
embeddings = []
labels_list = []

model.eval()
```

```

with torch.no_grad():
    for imgs, labels in test_loader:
        feats = model.forward_features(imgs.to(device)) # avec timm
        embeddings.append(feats.cpu().numpy())
        labels_list.append(labels.numpy())

embeddings = np.concatenate(embeddings)
labels_array = np.concatenate(labels_list)

# Projection t-SNE
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
embeddings_2d = tsne.fit_transform(embeddings)

# Visualisation
plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1],
            c=labels_array, cmap='coolwarm', alpha=0.6)
plt.title('t-SNE - EfficientNetV2 (Rouge = Malin, Bleu = Bénin)')
plt.colorbar()

```

Faire la comparaison pour les deux modèles et discuter lequel sépare mieux les classes.

☑ Checklist finale avant rendu

Obligatoire

- ☐ Audit du dataset documenté (distribution par grossissement et sous-types, visualisations)
- ☐ Split unique par patient, stratifié, seeds fixées — jamais retouché
- ☐ Déséquilibre traité (WeightedSampler + WeightedLoss, comparé à une baseline naïve)
- ☐ Métriques pertinentes : Recall, F1-macro, PR-AUC — pas seulement l'accuracy
- ☐ Fine-tuning documenté pour les deux architectures (courbes train/val, learning rates)
- ☐ Early stopping sur `val_loss`, checkpoint sauvegardé
- ☐ Analyse par grossissement (métriques + matrice de confusion par niveau)
- ☐ Grad-CAM sur EfficientNetV2 (≥5 images commentées, faux négatifs inclus)
- ☐ Attention Maps sur ViT (≥5 images commentées)
- ☐ Discussion des erreurs (quels cas sont difficiles, pourquoi)
- ☐ Notebooks propres et commentés, README complet, repo GitHub public

Bonus

- ☐ t-SNE / UMAP comparatif entre les deux architectures
- ☐ Focal Loss implémentée et comparée à la Weighted CE
- ☐ Classification multi-classes (8 sous-types tumoraux)
- ☐ Matrice de confusion 8×8 avec analyse des confusions inter-classes
- ☐ `requirements.txt` avec versions exactes

🧠 Pour mémoriser les concepts clés

Concept

Analogie simple

Concept	Analogie simple
Data leakage	Donner les réponses à un étudiant avant l'examen — les résultats sont biaisés
Transfer Learning	Un médecin généraliste qui se spécialise : il ne repart pas de zéro
Recall vs Precision	Recall = ne rater aucun cancer / Precision = ne pas affoler les saines
Grad-CAM	Demander au modèle "montre-moi où tu regardes" sur l'image
Attention Maps (ViT)	Le Transformer "surligne" les patches qui lui semblent importants
t-SNE	Projeter des données en 4D sur une feuille 2D — voir si les groupes se séparent

Projet réalisé dans le cadre du Master Expert en Intelligence Artificielle & Data — La Plateforme · 2026